
Oliver Krauss

Modern Code - Methods & Modularity I

ModernCode | MTD.ba VZ SS25

Hagenberg 27.10.2025

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Methods & Modularity I

- **Why methods?** Name the idea → reuse it, test it, reason about it
- **The problem:** Without methods, code becomes a tangled mess
 - Copy-paste everywhere leads to bugs and maintenance nightmares
 - Long, unreadable functions that do too many things
 - Impossible to test individual pieces of logic
 - Changes require updating code in multiple places
- **The solution:** Break complex problems into named, testable pieces
 - Each method has one clear responsibility
 - Reuse the same logic without duplicating code
 - Test each piece independently
 - Understand and modify code with confidence
- **Real-world analogy:** Like building with LEGO blocks instead of carving from stone

Method Syntax Overview

- **Basic structure:** `modifiers returnType name(params) { body }`
- **Components breakdown:**
 - **modifiers:** access control and behavior (public, private, static, final)
 - **returnType:** what the method gives back (void, int, double, String, etc.)
 - **name:** what to call the method (verb-based, descriptive)
 - **params:** what the method needs to work (input data)
 - **body:** the actual code that does the work
- **Example:** `public static int computeDamage(int base, double crit)`

Modifiers: Access Control & Behavior

- **Access modifiers:** control who can call the method
 - **public:** anyone can call this method (most permissive)
 - **private:** only code in the same class can call this method
 - **protected:** only code in the same package or subclasses can call
 - **(default):** only code in the same package can call (no modifier written)
- **Behavior modifiers:** change how the method works
 - **static:** method belongs to the class, not to instances
 - **final:** method cannot be overridden by subclasses
 - **synchronized:** method is thread-safe (advanced topic)
- **In this course:** We'll use **public static** for most methods

Note: We'll explore access modifiers, inheritance, and advanced OOP concepts in detail in Semester 2.

Static Methods: Class-Level Behavior

- **What static means:** The method belongs to the class itself, not to any specific object
- **How to call:** `ClassName.methodName()` or just `methodName()` if in the same class
- **Why we use it:**
 - No need to create objects to use the method
 - Perfect for utility functions and procedural programming
 - Methods can be called from anywhere without instantiation
- **Example:** `Math.sqrt(25)` - you don't need a Math object, just call the method directly
- **Limitation:** Static methods cannot access non-static fields or methods (they don't have access to object state)

Return Types: What Methods Give Back

- **Return types** specify what kind of value a method provides back to the caller
- **Three main categories:**
 - **void**: no return value (method just does something)
 - **Primitive types**: simple values like numbers and booleans
 - **Reference types**: objects and arrays
- **Key rule**: Non-void methods **MUST** return a value in all possible code paths

```
1  static int computeDamage(int base, double crit) {  
2      return (int) Math.round(base * crit);  
3  }
```

Return Type: void

- **void** means the method doesn't return any value
- **Use for:** Actions and operations that don't need to give back data
- **Common examples:**
 - Printing output: `IO.println()`
 - Updating game state: `updatePlayerHealth()`
 - Performing calculations without returning results

```
1  static void printStatus(int hp, int enemy) {  
2      IO.println("HP:" + hp + " | Enemy:" + enemy);  
3  }
```

Return Type: Primitive Types

- **Primitive types** return simple, basic values
- **Common primitive return types:**
 - **int**: whole numbers (damage, health points, scores)
 - **double**: decimal numbers (ratios, percentages, coordinates)
 - **boolean**: true/false values (isAlive, canAttack, isValid)
 - **char**: single characters (player initials, symbols)

```
1  static int computeDamage(int base, double crit) {  
2      return (int) Math.round(base * crit);  
3  }  
4  static boolean isPlayerAlive(int hp) {  
5      return hp > 0;  
6  }
```

Return Type: Reference Types

- **Reference types** return objects, arrays, or complex data structures
- **Common reference return types:**
 - **String**: text data (player names, status messages, descriptions)
 - **int[]**: arrays of numbers (inventory items, scores, coordinates)
 - Custom classes: complex data you create (Player, Item, GameState)

```
1  static String getPlayerStatus(int hp, int mana) {  
2      return "HP: " + hp + ", Mana: " + mana;  
3  }  
4  
5  static int[] getPlayerStats() {  
6      return new int[]{100, 50, 25}; // HP, Mana, XP  
7  }
```

Method Naming: Clear & Descriptive

- **Good method names** make code self-documenting and easy to understand
- **Three key principles:**
 - **Verb-first convention:** Methods do things, so start with action words
 - **Naming best practices:** Use clear, descriptive names that follow conventions
 - **Common patterns:** Standard naming conventions for common operations
- **Goal:** Make your code read like English sentences
- **Example:** `isPlayerAlive()` reads as "is player alive?"

Naming: Verb-First Convention

- **Methods do things, so start with action words**
- **Why verb-first?**
 - Makes it clear what the method will do
 - Code reads naturally: `computeDamage()` reads as "compute damage"
 - Avoids confusion with variable names (which are usually nouns)
- **Good examples:**
 - `computeDamage()` - calculates and returns damage
 - `renderStatus()` - displays status information
 - `validateInput()` - checks if input is valid
 - `updateScore()` - modifies the score
- **Poor examples:**
 - `damage()` - unclear what it does
 - `status()` - could be getting or setting status
 - `input()` - ambiguous purpose

Naming: Best Practices

- **Use camelCase:** Start with lowercase, capitalize each subsequent word
 - `calculateTotalDamage()` not `calculate_total_damage()`
 - `getPlayerHealth()` not `get_player_health()`
 - `isEnemyAlive()` not `is_enemy_alive()`
- **Be specific:** Avoid vague, generic names
 - `computeDamageWithCrit()` is better than `compute()`
 - `validatePlayerInput()` is better than `validate()`
 - `updateEnemyHealth()` is better than `update()`
- **Avoid abbreviations:** Use full words for clarity
 - `calculateDamage()` not `calcDmg()`
 - `getPlayerHealth()` not `getPlyrHp()`
 - `isPlayerAlive()` not `isPlyrAlv()`

Naming: Common Patterns

- **Standard naming conventions** for common operations
- **Getter methods:** Retrieve values
 - `get...()`: `getHealth()`, `getScore()`, `getName()`
 - `get...()` methods should not change the object's state
- **Setter methods:** Assign or update values
 - `set...()`: `setHealth(int hp)`, `setScore(int score)`
 - `set...()` methods change the object's state
- **Boolean methods:** Check conditions or states
 - `is...()`: `isAlive()`, `isValid()`, `isComplete()`
 - `has...()`: `hasWeapon()`, `hasPermission()`, `hasItems()`
 - `can...()`: `canAttack()`, `canMove()`, `canUse()`

Parameters: Input Data for Methods

- **Parameters** are data that methods need to do their job
- **Key concepts:**
 - **Syntax:** How to declare parameters correctly
 - **Types:** What kinds of data can be passed
 - **Pass-by-value:** How Java handles parameter passing
 - **Naming:** How to choose clear parameter names
- **Why parameters matter:** They make methods flexible and reusable
- **Example:** `computeDamage(10, 1.5)` passes 10 and 1.5 to the method

Method Signatures I

- **Method signature** is the method's unique identifier
- **Components:** method name + parameter types (in order)
 - The return type is not part of the signature
 - The parameter names are not part of the signature
- **What it defines:**
 - How to call the method
 - What data types to pass
 - What the method returns
 - How Java distinguishes between overloaded methods

Method Signatures II

Example: `computeDamage(int, double)` is the signature

```
1  // Method signature: computeDamage(int, double)
2  static int computeDamage(int base, double crit) {
3      return (int) Math.round(base * crit);
4  }
5
6  // Method signature: computeDamage(int)
7  static int computeDamage(int base) {
8      return computeDamage(base, 1.0); // calls the 2-param version
9  }
```

Note: Return type is NOT part of the signature, but parameter types and order are!

Method Overloading

- Multiple methods with same name but different parameters
- Java chooses the best match based on argument types
- Useful for providing convenience methods with sensible defaults
- Avoids having to remember different method names for similar functionality

Overloading Examples

```
1    static int computeDamage(int base) {
2        return computeDamage(base, 1.0); // call the 2-param version
3    }
4
5    static int computeDamage(int base, double crit) {
6        return (int) Math.round(base * crit);
7    }
8
9    // Usage examples:
10   int dmg1 = computeDamage(5);           // uses 1-param version
11   int dmg2 = computeDamage(5, 1.2);      // uses 2-param version
```

JVM Memory Areas: Where Data Lives

- **Java runs inside the JVM** which organizes memory into distinct areas
- **Method Area:** Permanent storage for class information
 - Class definitions, method code, static variables
 - Constants and metadata
- **Stack:** Temporary storage for method execution
 - Method calls, parameters, local variables
 - Each method gets its own "frame"
- **Heap:** Dynamic storage for objects
 - Object instances, arrays, dynamic data
 - Managed by garbage collection
- **PC Register:** Points to the current instruction being executed
- **Native Method Stack:** For native methods (code written in other languages)

The Stack: Method Call Organization

- **Stack structure:** Last In, First Out (LIFO) - like a stack of plates
- **Stack frame:** Each method call gets its own "plate" containing:
 - Method parameters (copied from caller)
 - Local variables declared in the method
 - Return address (where to go when method finishes)
 - Temporary calculation results
- **Automatic management:** Frames are created when methods start, destroyed when they end
- **Characteristics:** Limited size, very fast access, no fragmentation, automatic cleanup

Method Call Stack: Step by Step

- **Call chain:** `main()` → `process()` → `helper()`
- **What happens:**
 - Each method call creates a new stack frame
 - Parameters are copied from caller's frame to callee's frame
 - Local variables are created in the new frame
 - When method returns, its frame is destroyed
- **Stack depth:** Maximum number of frames = maximum call depth
- **Why it matters:** Stack overflow happens when call chain is too deep

Stack Simple Example

```
1  static int helper(int x) {  
2      int temp = x * 2;  
3      return temp;  
4  }  
5  static int process(int a, int b) {  
6      int res = helper((a + b));  
7      return res;  
8  }  
9  static void main() {  
10     int num1 = 5;  
11     int num2 = 3;  
12     int num3 = process(num1, num2);
```

Stack Variables Available

Method	Variables
--------	-----------

Each method call creates a new frame with its own variables

Stack Simple Example

```
1  static int helper(int x) {  
2      int temp = x * 2;  
3      return temp;  
4  }  
5  static int process(int a, int b) {  
6      int res = helper((a + b));  
7      return res;  
8  }  
9  static void main() {  
10     int num1 = 5;  
11     int num2 = 3;  
12     int num3 = process(num1, num2);
```

Stack Variables Available

main()	num1=5, num2=3, num3=?
Method	Variables

Each method call creates a new frame with its own variables

Stack Simple Example

```
1  static int helper(int x) {  
2      int temp = x * 2;  
3      return temp;  
4  }  
5  static int process(int a, int b) {  
6      int res = helper((a + b));  
7      return res;  
8  }  
9  static void main() {  
10     int num1 = 5;  
11     int num2 = 3;  
12     int num3 = process(num1, num2);
```

Stack Variables Available

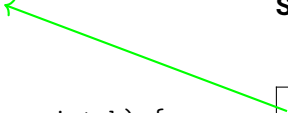
process()	a=5, b=3, res=?
main()	num1=5, num2=3, num3=?
Method	Variables

Each method call creates a new frame with its own variables

Stack Simple Example

```
1  static int helper(int x) {  
2      int temp = x * 2;  
3      return temp;  
4  }  
5  static int process(int a, int b) {  
6      int res = helper((a + b));  
7      return res;  
8  }  
9  static void main() {  
10     int num1 = 5;  
11     int num2 = 3;  
12     int num3 = process(num1, num2);
```

Stack Variables Available



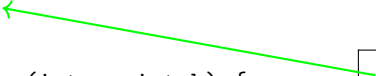
helper()	x=8, temp=?
process()	a=5, b=3, res=?
main()	num1=5, num2=3, num3=?
Method	Variables

Each method call creates a new frame with its own variables

Stack Simple Example

```
1  static int helper(int x) {  
2      int temp = x * 2;  
3      return temp;  
4  }  
5  static int process(int a, int b) {  
6      int res = helper((a + b));  
7      return res;  
8  }  
9  static void main() {  
10     int num1 = 5;  
11     int num2 = 3;  
12     int num3 = process(num1, num2);
```

Stack Variables Available



helper()	x=8, temp=16
process()	a=5, b=3, res=?
main()	num1=5, num2=3, num3=?
Method	Variables

Each method call creates a new frame with its own variables

Stack Simple Example

```
1  static int helper(int x) {  
2      int temp = x * 2;  
3      return temp;  
4  }  
5  static int process(int a, int b) {  
6      int res = helper((a + b));  
7      return res;  
8  }  
9  static void main() {  
10     int num1 = 5;  
11     int num2 = 3;  
12     int num3 = process(num1, num2);
```

Stack Variables Available

process()	a=5, b=3, res=16
main()	num1=5, num2=3, num3=?
Method	Variables

Each method call creates a new frame with its own variables

Stack Simple Example

```
1  static int helper(int x) {  
2      int temp = x * 2;  
3      return temp;  
4  }  
5  static int process(int a, int b) {  
6      int res = helper((a + b));  
7      return res;  
8  }  
9  static void main() {  
10     int num1 = 5;  
11     int num2 = 3;  
12     int num3 = process(num1, num2);
```

Stack Variables Available

main()	num1=5, num2=3, num3=16
Method	Variables

Each method call creates a new frame with its own variables

Stack Overflow: When the Stack Gets Too Deep

- **Stack overflow** happens when method calls create too many stack frames
- **Common causes:**
 - Infinite recursion (method calling itself forever)
 - Very deep call chains
 - Recursive algorithms without proper base cases
- **What happens:** JVM runs out of stack memory and crashes

```
1  static void infiniteCall() {  
2      IO.println("Calling...");  
3      infiniteCall();  // Calls itself forever!  
4  }  
5  // This will eventually cause:  
6  // Exception in thread "main" java.lang.StackOverflowError
```

The Heap: Object Storage

- **Heap structure:** Large, flexible memory area for objects
- **What lives here:**
 - All objects created with **new** keyword
 - Arrays (both primitive and object arrays)
 - String objects and other reference types
 - Large data structures
- **Memory management:** Garbage collector frees unused objects
- **Access:** Through references (memory addresses) stored in variables
- **Characteristics:** Much larger than stack, can grow/shrink, potential fragmentation, slower access

Stack vs Heap: Visual Example

Stack Variables

Variable	Value
primitive	42
text	0x1234
numbers	0x5678

Heap Objects

Address	Object
0x1234	String: "Hello"
0x5678	Array: [1, 2, 3]

```
1 static void example() {  
2     int primitive = 42;  
3     String text = "Hello";  
4     int[] numbers = {1, 2, 3};  
5 }  
6
```

Why Understanding Memory Matters

- **Debugging:** Stack traces show the call chain and frame information
- **Performance:** Stack access is faster than heap access
- **Memory leaks:** Objects on heap that aren't referenced can't be garbage collected
- **Stack overflow:** Infinite recursion or very deep calls can crash the program
- **Data sharing:** References allow multiple methods to work with the same object
- **Scope:** Local variables are isolated to their method frame
- **Parameter passing:** Understanding why primitives vs references behave differently

Parameters, Returns, Pass-by-Value

- Java is **pass-by-value** (arguments copied into parameters)
- **For primitives:** The value itself is copied - changes don't affect caller
- **For references:** The reference (memory address) is copied, but the object is shared
- Return a value to communicate results; avoid hidden state

Pass-by-Value: Primitives

Primitives are passed by value - the actual value gets copied

- Java creates a copy of the primitive value
- Method works with its own local copy
- Changes to the parameter don't affect the original variable

```
1  static void modifyPrimitive(int x) {  
2      x = x + 10;  // modifies local copy only  
3      IO.println("Inside method: " + x);  
4  }  
5  
6  int value = 5;  
7  modifyPrimitive(value);    // prints "Inside method: 15"  
8  IO.println("After call: " + value);  // still prints 5
```

Pass-by-Value: References

References are passed by value - the reference (memory address) gets copied

- Java creates a copy of the reference (memory address)
- Both the caller and method point to the same object
- Changes to the object affect both the caller and method
- The object itself is NOT copied

```
1  static void modifyArray(int[] arr) {
2      arr[0] = 40;  // modifies the shared array object
3      IO.println("Inside method: " + arr[0]);
4  }
5  int[] array = {2};
6  IO.println("Before: " + array[0]);  // prints 2
7  modifyArray(array);                // prints "Inside method: 40"
8  IO.println("After: " + array[0]);   // prints 40!
```

"Global Variables" in Java

- Java has no true globals; **static fields** act like them
- Prefer passing values via parameters; keep shared mutable state minimal
- Constants are OK: **public static final** constants (read-only)

Constants vs Mutable Globals

```
1    public class GameConfig {  
2        public static final int MAX_HEALTH = 100;    // OK: constant  
3        // public static int currentLevel = 1;      // Avoid: mutable  
4    }  
5
```

Use locals and returns; reserve class-level state for constants.

Using Constants Across Functions

```
1  public class Game {
2      public static final int MAX_HEALTH = 100;  // constant
           accessible to all methods
3      public static void main(String[] args) {
4          int playerHealth = 50;
5          IO.println(playerHealth + "/" + MAX_HEALTH);
6          checkHealth(playerHealth);
7      }
8      public static void checkHealth(int health) {
9          if (health > MAX_HEALTH) {
10             IO.println("Invalid health value!");
11         }
12     }
```

Scope: Where Names Are Visible

- **Scope** defines where a variable name can be used in your code
- **Local variables:** exist only within their {block}
 - Declared inside a method or loop
 - Cannot be accessed outside their block
 - Each method call gets its own copy
- **Parameters:** locals of the method
 - Receive values from the caller
 - Behave like local variables
 - Exist only while the method is executing
- **Block scope:** variables declared in {curly braces}
 - if statements, loops, try-catch blocks
 - Variables live only within those braces

Scope Examples: Bad Practices

- **Common scope mistakes:** Variables declared too broadly
- **Problems with this approach:**
 - Variables exist longer than needed
 - Can lead to confusion and bugs
 - Unnecessary memory usage

```
1  static void badScope() {  
2      int i, temp; // declared outside loop - unnecessary  
3      for (i = 0; i < 5; i++) {  
4          temp = i * 2;  
5          // i and temp still exist after loop - confusing!  
6      }  
7      // i = 5, temp = 8 - but we don't need these values!  
8  }
```

Scope Examples: Good Practices

-- **Good scope practices:** Variables live only where needed

-- **Benefits of this approach:**

- Clear variable purpose and lifetime
- Easier to understand and debug
- More efficient memory usage

```
1  static void goodScope() {  
2      int total = 0;  
3      for (int i = 0; i < 5; i++) { // i only exists in loop  
4          int temp = i * 2;        // temp only exists in loop  
5          total += temp;  
6      }  
7      // i and temp are not accessible here - clean!  
8      IO.println("Total: " + total);  
}
```

Variable Shadowing: The Problem

- **Shadowing** happens when inner scope reuses names from outer scope
- **Common in Java:** Local variables shadowing static (global) variables
- **Problem:** Same name, different variable - very confusing!
- **What happens:** Local variable hides the static variable

```
1  public class Game {
2      public static int MAX_HEALTH = 100;  // static (global) variable
3
4      public static void main(String[] args) {
5          int MAX_HEALTH = 50;  // local variable shadows static variable
6          IO.println(MAX_HEALTH);  // prints 50 (local)
7          // How do we access the static MAX_HEALTH = 100?
8      }
9  }
```

Variable Shadowing: The Solution

- **Solution 1:** Use the class name to access static variables
- **Solution 2:** Use unique names to avoid shadowing
- **Best practice:** Choose descriptive names that don't conflict

```
1  public class Game {
2      public static int MAX_HEALTH = 100;  // static (global) variable
3
4      public static void main(String[] args) {
5          // Solution 1: Use class name to access static variable
6          IO.println("Max: " + Game.MAX_HEALTH);
7          // Solution 2: Use unique names
8          int playerHealth = 50;           // no shadowing
9          IO.println("Player: " + playerHealth + ", Max: " + MAX_HEALTH);
```

Scope Best Practices

- **Declare variables close to where they're used**
 - Don't declare all variables at the top of a method
 - Declare loop variables in the loop header
 - Declare temporary variables just before use
- **Keep variable lifetimes short**
 - Variables should exist only as long as needed
 - Shorter lifetimes = easier to understand and debug
 - Reduces chance of bugs from stale data
- **Use descriptive names**
 - Avoid single letters except for simple counters
 - Names should indicate purpose and scope
- **Be careful with nested scopes**
 - Each level of nesting can have its own variables
 - Avoid deep nesting (more than 3-4 levels)
 - Consider extracting complex nested logic into methods

Method Design Basics

- **Single Responsibility**: One clear purpose per method
- **Verb-based names**: Methods do things, so start with action words
- **Return values**: Prefer returning results over mutating shared state
- **Parameters**: Keep lists small and meaningful
- **Constants**: Use for configuration values, not magic numbers

Method Design: Good Examples

- **Clear purpose:** Each method has one job
- **Descriptive names:** Names explain what the method does
- **Small parameters:** Easy to understand and test

```
1  // Good: Clear purpose, descriptive names
2  static int computeDamage(int baseDamage, double criticalMultiplier)
3      {
4      return (int) Math.round(baseDamage * criticalMultiplier);
5  }
6
7  static void renderPlayerStatus(int health, int maxHealth) {
8      IO.println("Health: " + health + "/" + maxHealth);
9  }
```

Method Design: Poor Examples

- **Unclear purpose:** Method does too many things
- **Vague names:** Names don't explain what happens
- **Too many parameters:** Hard to understand and test

```
1  // Poor: Unclear purpose, vague names, too many params
2  static void doStuff(int a, int b, int c, int d, boolean flag) {
3      // What does this method do? Unclear!
4      if (flag) {
5          IO.println(a + b);
6      } else {
7          IO.println(c + d);
8      }
9  }
```

Method Design Principles

- **Keep methods small:** Usually 5-15 lines
- **One level of abstraction:** Don't mix high and low-level operations
- **Descriptive names:** Method name should be a clear command
- **Minimal parameters:** 3-4 parameters maximum
- **Clear return values:** Return type should match the method's purpose
- **No side effects:** Method should do what its name promises

Refactoring: Breaking Down Complex Methods

- **Problem:** One big method doing too many things
- **Solution:** Break it into smaller, focused methods
- **Benefits:** Easier to understand, test, and maintain
- **Process:** Identify distinct operations, extract into separate methods

Refactoring: The Problem

```
1  static void processPlayerTurn(int playerHealth, int enemyHealth,
    int damage) {
2      // Calculate new health
3      int newEnemyHealth = Math.max(0, enemyHealth - damage);
4      // Check if enemy is dead
5      boolean enemyDead = newEnemyHealth <= 0;
6      // Display status
7      IO.println("Enemy health: " + newEnemyHealth);
8      if (enemyDead) {
9          IO.println("Enemy defeated!");
10     }
11 }
```

Refactoring: The Solution

```
1  static int applyDamage(int currentHealth, int damage) {
2      return Math.max(0, currentHealth - damage);
3  }
4  static boolean isDead(int health) {
5      return health <= 0;
6  }
7  static void displayStatus(int health, boolean isDead) {
8      IO.println("Enemy health: " + health);
9      if (enemyDead) {
10         IO.println("Enemy defeated!");
11     }
12 }
```

Refactoring: How to Use the New Methods

```
1  // Now the main method is clean and easy to understand
2  static void processPlayerTurn(int playerHealth, int enemyHealth,
    int damage) {
3      int newEnemyHealth = applyDamage(enemyHealth, damage);
4      boolean enemyDead = isDead(newEnemyHealth);
5      displayStatus(newEnemyHealth, enemyDead);
6  }
```

Common Method Mistakes

- **Missing return statements** in non-void methods
- **Over-reliance on static fields** instead of parameters/returns
- **Variable shadowing** and confusing local variable lifetimes
- **Unreachable code** after return statements
- **Parameter misuse** and incorrect data flow

Missing Return Statements: The Problem

- **Problem:** Non-void methods must return a value in all code paths
- **Compiler error:** "missing return statement"
- **What happens:** Code won't compile

```
1  // BAD: Missing return in some code paths
2  static int getDamage(int weaponType) {
3      if (weaponType == 1) {
4          return 10;
5      } else if (weaponType == 2) {
6          // Missing return for other weapon types!
7      }
8      return 0; // Default return for other cases
9  }
```

Missing Return Statements: The Solution

- **Solution:** Ensure every possible path has a return statement
- **Approach:** Use else clause or default return
- **Benefit:** Code compiles and handles all cases

```
1  // GOOD: All paths have return statements
2  static int getDamage(int weaponType) {
3      if (weaponType == 1) {
4          return 10;
5      } else if (weaponType == 2) {
6          return 15;
7      } else {
8          return 5; // Default damage
9      }
10 }
```

Over-using Static Fields: The Problem

- **Problem:** Using static fields as "global variables" for everything
- **Issues:** Hard to track state, difficult to test, hidden dependencies
- **What happens:** Code becomes hard to understand and debug

```
1  // BAD: Over-using static fields
2  public class Game {
3      public static int playerHealth = 100;  // Global state
4      public static int enemyHealth = 50;
5
6      public static void attack() {
7          enemyHealth -= 10;  // Modifies global state
8      }
9  }
```

Over-using Static Fields: The Solution

- **Solution:** Pass data via parameters and return values
- **Approach:** Methods take input and return output
- **Benefit:** Clear data flow, easier to test and debug

```
1  // GOOD: Pass data via parameters and returns
2  public class Game {
3      public static int attack(int enemyHealth, int damage) {
4          return Math.max(0, enemyHealth - damage); // Returns new
           state
5      }
6  }
```

Unreachable Code: The Problem

- **Problem:** Code after a return statement that can never execute
- **Compiler warning:** "unreachable code"
- **What happens:** Code is ignored, potential logic errors

```
1  // BAD: Unreachable code after return
2  static int getDamage(int weaponType) {
3      if (weaponType == 1) {
4          return 10;
5          IO.println("Weapon 1 selected"); // Never executes
6      }
7      return 5;
8  }
```

Unreachable Code: The Solution

- **Solution:** Move code before return statements
- **Approach:** Ensure all code can potentially execute
- **Benefit:** No warnings, all logic is reachable

```
1  // GOOD: All code is reachable
2  static int getDamage(int weaponType) {
3      if (weaponType == 1) {
4          IO.println("Weapon 1 selected"); // Executes first
5          return 10;                       // Then returns
6      }
7      return 5;
8  }
```

Parameter Misuse: The Problem

- **Problem:** Parameters used incorrectly or in wrong order
- **Issues:** Wrong results, hard to debug, confusing code
- **What happens:** Method works but produces unexpected output

```
1  // BAD: Parameters used in wrong order
2  static void displayStatus(int health, int maxHealth) {
3      IO.println("Health: " + maxHealth + "/" + health); // Wrong
4      order!
5  }
6  // BAD: Parameter ignored or misused
7  static int calculateDamage(int baseDamage, int criticalMult) {
8      return baseDamage * 2; // Ignores criticalMult parameter!
9  }
```

Parameter Misuse: The Solution

- **Solution:** Use parameters correctly and in proper order
- **Approach:** Match parameter usage to their intended purpose
- **Benefit:** Correct results, clear logic, easy to debug

```
1  // GOOD: Parameters used correctly
2  static void displayStatus(int health, int maxHealth) {
3      IO.println("Health: " + health + "/" + maxHealth);
4  }
5  // GOOD: All parameters used appropriately
6  static int calculateDamage(int baseDamage, int criticalMult) {
7      return baseDamage * criticalMult;
8  }
```

How to Avoid Common Mistakes

- **Plan your method** before writing code
 - What should it return? What type?
 - What parameters does it need?
 - What are all the possible code paths?
- **Test your methods** with simple examples
 - Test edge cases (0, negative numbers, empty inputs)
 - Check that all code paths work
- **Use the compiler** as your first line of defense
 - Fix all compiler errors before running
 - Pay attention to type mismatch warnings
 - Let the compiler catch basic mistakes
- **Keep methods simple** and focused
 - One clear purpose per method
 - Small parameter lists
 - Clear return values

AI Review: Method Design Analysis

- **The task:** Compare two versions of the same method
- **Goal:** Identify which design is better and why
- **Use AI tools:** ChatGPT, Copilot, or other AI assistants
- **Focus areas:** Clarity, maintainability, and best practices

AI Review: Example Method Variants

```
1  // Version A: Simple but limited
2  static int applyDamage(int targetHealth, int damage) {
3      return Math.max(0, targetHealth - damage);
4  }
5  // Version B: Complex but potentially confusing
6  static int processCombat(int hp, int dmg, int armor, boolean
7      isCritical, int level) {
8      int finalDamage = dmg;
9      if (isCritical) finalDamage *= 2;
10     if (level > 10) finalDamage += 5;
11     int actualDamage = Math.max(1, finalDamage - armor);
12     return Math.max(0, hp - actualDamage);
13 }
```

AI Review: Analysis Criteria

- **Clarity of purpose:** What does the method do?
 - Is the name descriptive and accurate?
 - Does the method have a single, clear responsibility?
 - Is the logic easy to follow?
- **Input/Output design:** How does data flow?
 - Are parameters named clearly?
 - Is the parameter order logical?
 - Does the return type match the purpose?
- **Maintainability:** How easy is it to modify?
 - Are there magic numbers?
 - Is the code well-structured?
 - Are there hidden dependencies?

AI Review: Prompt Template

Prompt structure for consistent analysis:

- **Compare the two versions** of [method name]:
 - Which version is clearer and why?
 - What are the strengths and weaknesses of each?
 - Suggest a better method name
 - Suggest better parameter names and order
 - Identify any magic numbers or unclear logic
- **Ask for specific improvements** and code examples
- **Request explanations** for why certain choices are better

AI Review: Expected Outcomes

- **Better method names:** More descriptive and action-oriented
- **Improved parameters:** Clear names and logical order
- **Code structure:** Cleaner, more maintainable design
- **Best practices:** Following established conventions
- **Learning insights:** Understanding why certain designs are better

Remember: AI is a tool to help you think critically about code design. Always verify and understand the suggestions it provides.

Summary: Methods & Modularity I

Method Foundations

- Syntax: modifiers, return types, names, parameters
- Procedural programming
- Method signatures and overloading basics

Memory & Execution Model

- JVM memory: Stack, Heap, Method Area
- Stack frames, local variables, method calls
- Pass-by-value: primitives vs references

Scope & Design Principles

- Variable scope: local, parameters, constants
- Single responsibility principle
- Refactoring complex methods into smaller ones

Common Mistakes & Best Practices

- Return statements, type mismatches, unreachable code
- Parameter misuse, static field overuse
- Clear naming, proper parameter design

AI-Assisted Code Review

- Using AI tools to analyze method design
- Criteria for evaluating code quality
- Improving clarity and maintainability

Key Takeaways

- Methods break complex problems into manageable pieces
- Good design leads to maintainable code
- AI tools enhance learning when used thoughtfully